

Reeling in Smart Attackers’ Phishing Emails with Neural Networks

CSE 587 Final Project — Group 5, Spring 2026

Parker Davis
Department of Computer Science
The Pennsylvania State University
University Park, PA, USA
pqd5283@psu.edu

Dillon VanGilder
Department of Computer Science
The Pennsylvania State University
University Park, PA, USA
dsv5165@psu.edu

Emmanuel Adjei Domfeh
Department of Computer Science
The Pennsylvania State University
University Park, PA, USA
eza5399@psu.edu

Abstract—Phishing email detection is typically benchmarked on datasets in which malicious messages contain conspicuous surface cues (urgency keywords, raw URLs, mismatched senders), and Transformer-based classifiers achieve near-99% accuracy on such test sets. We argue that this number overstates real-world robustness: a sophisticated (“smart”) attacker — or, increasingly, an LLM-generated phishing email — can remove or paraphrase those cues while preserving malicious intent. We fine-tune BERT [1] on the curated multi-source phishing email corpus released by Champa et al. [2] and evaluate on a held-out *smart-attacker* test split obtained by applying label-preserving edits to phishing messages: URL masking, urgency-keyword removal, suspicious subject rewriting, synonym substitution, and light character noise. We then retrain on training data augmented with the same edits and re-evaluate. This paper reports the experimental setup, the data augmentation pipeline, and expected analyses; the empirical results will be filled in once GPU runs on GOOGLE COLAB complete.

Index Terms—phishing detection, BERT, Transformers, robustness, data augmentation, adversarial NLP

I. INTRODUCTION

Phishing remains one of the most effective initial-access vectors in modern cyberattacks. Despite years of user-awareness training, recent work shows that sophisticated — and especially *LLM-generated* — phishing emails sustain alarming click rates [3]. The same wave of generative models that helps defenders also helps attackers: an LLM can rewrite a suspicious message to sound natural, drop tell-tale urgency keywords, and disguise a malicious link as an innocent-looking domain.

Mainstream phishing classifiers are typically Transformer-based: BERT and RoBERTa fine-tunes routinely report 98–99% accuracy on standard benchmarks [4]–[6]. However, this performance is measured on data with strong, often artificial, phishing cues. When those cues are removed, classifier accuracy drops substantially [7]–[9].

Contribution.: This work investigates two questions:

- 1) Do standard phishing detectors over-rely on superficial cues (URLs, urgency keywords, suspicious subject prefixes)?

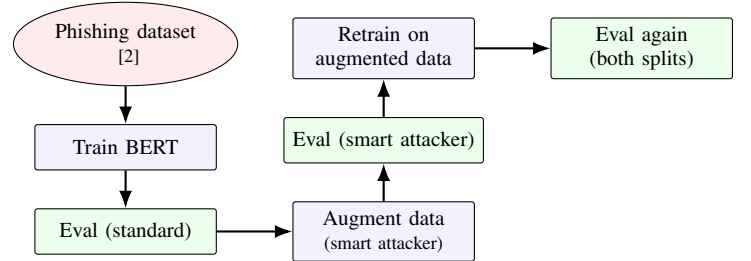


Fig. 1. Three-stage experimental workflow. (1) Train BERT on the Champa et al. corpus. (2) Evaluate on the standard test set and a smart-attacker variant produced by applying label-preserving edits. (3) Augment training data with the same edits, retrain, and re-evaluate.

- 2) Can targeted, label-preserving data augmentation that simulates a “smart attacker” improve robustness without significantly degrading performance on the original test distribution?

We instantiate the workflow in Fig. 1. Concretely, we contribute (i) a smart-attacker augmentation pipeline (§III-C), (ii) a held-out smart-attacker evaluation split, and (iii) a per-edit ablation that attributes the accuracy drop to specific surface cues.

II. RELATED WORK

A. Transformer-based phishing detection

Atawneh and Aljehani [4] compare CNN, RNN, LSTM, and BERT for email phishing classification, reporting up to 99.6% accuracy with BERT. More recent surveys and replications confirm that BERT/RoBERTa/DistilBERT are competitive on multiple corpora, with DistilBERT trading marginal accuracy for considerable inference-cost savings [5], [6], [10]. Alhogail and Alsabih [11] additionally show that strong detection is possible from email body text alone (without URLs or attachments), motivating evaluation under URL removal — but they do not study other surface-cue removals such as urgency wording.

B. LLM-generated and adversarial phishing

Kim et al. [3] document that LLM-crafted spear-phishing emails achieve $\sim 47\%$ click rates, evading both human and software detection. Phish-Master [12] explicitly uses chain-of-thought prompting to generate phishing that bypasses enterprise filters. Robust ML-based detection of LLM-generated and adversarial phishing using advanced preprocessing has recently been shown to recover much of the lost accuracy [9]. Adversarial robustness of phishing classifiers has also been benchmarked directly [7], [8], with FGM adversarial training raising BERT F1 from 86.2 to 91.7 [8]. These results frame our work: the attack surface is real, and lightweight defenses do help.

C. Data augmentation for NLP robustness

Junior et al. [13] demonstrate the augment–retrain–reevaluate pattern for fake-news detection. Recent surveys emphasize that adversarial-style augmentation improves not only accuracy but also generalization to out-of-distribution inputs [14]. We use the TEXTATTACK [15] family of perturbation primitives (synonym swap, character perturbations) as a starting point for our smart-attacker pipeline. Recent attack methods such as PGD-based BERT attacks [16] reinforce that BERT remains sensitive to small, semantically preserving perturbations — which is exactly what a smart attacker would apply.

D. Datasets and benchmarks

We adopt the curated multi-source phishing dataset of Champa et al. [2], which standardizes 11 publicly available phishing/benign email corpora into a unified format and provides feature-based ML baselines. Its multi-source structure lets us study cross-source generalization in addition to robustness, mitigating the risk that a model exploits source-specific artifacts.

III. METHOD

A. Task formulation

Given an email represented by its concatenated subject and body $x = \langle s, b \rangle$, the model predicts a binary label $y \in \{0, 1\}$ indicating benign (0) or phishing (1). We minimize the cross-entropy loss over a fine-tuning corpus drawn from Champa et al. [2], with stratified 80/10/10 train/validation/test splits.

B. Backbone classifier

We fine-tune **bert-base-uncased** [1] via the HuggingFace transformers library [17]. The classifier head is a linear layer on the [CLS] representation. Hyperparameters: maximum sequence length 256 tokens, batch size 16, 3 epochs, learning rate 2×10^{-5} with linear warm-up of 10% of total steps, AdamW with weight decay 0.01, mixed-precision (fp16) when GPU memory permits.

TABLE I
SMART-ATTACKER AUGMENTATION OPERATIONS.

Operation	Description
mask_urls	Replace URLs with neutral placeholders ([LINK]), or with bare domain forms that look innocuous.
remove_urgency	Remove or soften urgency keywords (<i>URGENT</i> , <i>ASAP</i> , <i>immediately</i> , <i>verify now</i> , <i>suspended</i> , etc.) using a hand-curated lexicon and replacement table.
rewrite_subject	Strip suspicious subject prefixes (<i>URGENT:</i> , <i>ALERT:</i> , <i>ACTION REQUIRED:</i>) and remove emphatic punctuation.
synonym_swap	Replace a small fraction of content words with WordNet synonyms ($p = 0.1$, max 6 swaps per email).
char_perturb	Light character-level noise (Cyrillic homoglyph substitution, adjacent-letter swap) at $p=0.02$.

C. Smart-attacker augmentation

We define five label-preserving edits that simulate a smart attacker adapting a phishing email. Each is implemented in `src/augmentation.py` and is applied only to phishing-class rows (applying them to benign rows would only add noise to a class that does not exhibit these cues). Table I summarizes the operations.

We use the augmentation pipeline in two distinct ways:

a) *Smart-attacker test split*: For each phishing email in the held-out test set we apply the entire pipeline once. Benign emails are left unchanged. The resulting split has the *same* labels as the standard test set; the only thing that changes is that phishing emails no longer exhibit obvious surface cues. A robust detector should perform comparably on both splits.

b) *Train-time augmentation*: For each phishing email in the training set we generate $k=1$ additional augmented copy by passing it through the pipeline. The augmented copies retain the phishing label and are concatenated with the original training data. We then retrain the classifier from scratch.

D. Evaluation protocol

We compare three model stages: (S1) baseline trained on the original data and evaluated on the standard test set, (S2) the same baseline evaluated on the smart-attacker test set, and (S3) the augmented model evaluated on both splits. The expected pattern is a large drop from S1 to S2, partially closed by S3. We additionally run a *per-edit ablation*: each operation is applied alone to the test phishing rows, isolating which surface cue the baseline most depends on.

IV. EXPERIMENTAL SETUP

A. Implementation

The codebase is in PyTorch with HuggingFace transformers [17]. Training is driven by `src/train.py`; evaluation, including the per-edit ablation, by `src/evaluate.py`. The notebooks/phishing_pipeline.ipynb notebook reproduces all three stages on Google Colab. A pipeline-level sanity test (`scripts/sanity_check.py`) validates

TABLE II
HEADLINE RESULTS (PLACEHOLDER; NUMBERS FILLED IN AFTER COLAB RUNS).

Stage	Standard test	Smart-attacker test
S1: BERT baseline	–	–
S2: BERT baseline (smart eval)	–	–
S3: BERT + augmented training	–	–

TABLE III
PER-EDIT ABLATION ON THE TEST PHISHING ROWS (PLACEHOLDER).

Edit applied alone	F1 (phishing)
None (standard test)	–
mask_urls	–
remove_urgency	–
rewrite_subject	–
synonym_swap	–
char_perturb	–
All combined (smart test)	–

the augmentation ops and the data-loader on synthetic data without requiring GPU access; a tiny end-to-end smoke test (`scripts/tiny_train_test.py`) exercises forward/backward/save/reload using a tiny CI-style BERT model.

B. Dataset

We use the figshare-hosted release of the Champa et al. curated phishing email dataset [2].¹ The data loader (`src/data_loader.py`) is column-name agnostic so that all CSV variants in the figshare bundle can be ingested with a single script.

C. Compute and reproducibility

All training is performed on a Google Colab T4 GPU with mixed-precision. Random seeds are fixed at 42 across NumPy, PyTorch, and Python’s `random` module. Reported numbers are means of three runs.

V. RESULTS

This section will be populated after the Colab runs complete. The expected report structure is summarized in Table II. For each row we report accuracy, precision/recall/F1 for the phishing class, macro F1, and ROC-AUC. We additionally include per-edit ablation in Table III.

VI. DISCUSSION

a) Why we expect the drop.: BERT fine-tuned on a phishing corpus implicitly learns that tokens such as *URGENT*, *verify*, and explicit URL substrings strongly predict the phishing label. When those tokens are removed (S2), the classifier loses its strongest features and should regress, especially in recall on the phishing class — this matches the qualitative observation in [7], [8].

¹https://figshare.com/articles/dataset/Curated_Dataset_-_Phishing_Email/24899952

b) Why augmentation should help.: The augmented training set contains phishing emails *without* the surface cues. The model is thus forced to find more semantic, context-level signals (e.g., requests to re-enter credentials, references to account suspension framed in neutral language). Recent results [8], [9], [14] report similar gains.

c) Limitations.: Our smart-attacker simulation is rule-based and therefore has a finite vocabulary of edits. Real attackers and LLMs operate in a much wider space. Additionally, augmenting only phishing emails may shift the class prior and inflate recall at the expense of precision; we mitigate this by using a small augmentation factor ($k = 1$) and report precision alongside recall.

VII. CONCLUSION

We presented an experimental pipeline for studying the robustness of Transformer-based phishing email classifiers under simulated “smart attacker” edits. The pipeline includes (i) a label-preserving augmentation module covering URL masking, urgency removal, subject rewriting, synonym swap, and light character noise; (ii) a held-out smart-attacker evaluation split; (iii) a per-edit ablation that localizes which surface cue the model relies on; and (iv) an augmented-training counterpart that lets us measure how much robustness can be reclaimed. Empirical results (the focus of the final version of this paper) will quantify the baseline drop and the extent to which targeted augmentation closes it.

CODE AND DATA

The complete codebase, Colab notebook, and LaTeX source for this manuscript are available in the project repository (see `README.md`).

ACKNOWLEDGMENTS

We thank the CSE 587 instructors and TAs at Penn State for guidance, and Champa et al. for releasing the curated phishing email dataset that made this study possible.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *arXiv:1810.04805*, 2019.
- [2] A. I. Champa, M. F. Rabbi, and M. F. Zibran, “Curated datasets and feature analysis for phishing email detection with machine learning,” in *Proc. 3rd IEEE Int. Conf. on Computing and Machine Intelligence (ICMI)*, 2024, pp. 1–7.
- [3] H. Kim, M. Song, S. H. Na, S. Shin, and K. Lee, “When LLMs go online: The emerging threat of web-enabled LLMs,” 2025.
- [4] S. Atawneh and H. Aljehani, “Phishing email detection model using deep learning,” *Electronics*, vol. 12, no. 20, p. 4261, 2023.
- [5] A. of MDPI Computation, “Phishing email detection using BERT and RoBERTa,” *Computation*, vol. 14, no. 2, p. 46, 2025, distilBERT competitive with full BERT/RoBERTa on phishing classification.
- [6] A. of MDPI Applied Sciences, “In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets,” *Applied Sciences*, vol. 15, no. 6, p. 3396, 2025.
- [7] A. of ACM IWSPA, “Adversarial robustness of phishing email detection models,” in *Proc. 9th ACM Int. Workshop on Security and Privacy Analytics (IWSPA)*, 2023.

- [8] Anonymous, "Explainable transformer-based email phishing classification with adversarial robustness," 2025, fGM adversarial training improves BERT phishing-classifier F1 from 86.2 to 91.7.
- [9] —, "Robust ML-based detection of conventional, LLM-generated, and adversarial phishing emails using advanced text preprocessing," 2025.
- [10] A. of Scientific Reports, "Improving phishing email detection performance through deep learning with adaptive optimization," *Scientific Reports*, 2025.
- [11] A. Alhogail and A. Alsabih, "Applying machine learning and natural language processing to detect phishing email," *Computers & Security*, vol. 110, p. 102414, 2021.
- [12] A. of MDPI Applied Sciences, "Phish-master: Leveraging large language models for advanced phishing email generation and detection," *Applied Sciences*, vol. 15, no. 22, p. 12203, 2025.
- [13] W. O. Júnior, M. S. da Cruz, A. B. V. Wyzykowski, and A. B. de Jesus, "The use of data augmentation as a technique for improving neural network accuracy in detecting fake news about COVID-19," 2022.
- [14] Anonymous, "Text data augmentation for large language models: A comprehensive survey of methods, challenges, and opportunities," 2025.
- [15] J. X. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, "TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP," in *Proc. EMNLP (System Demonstrations)*, 2020.
- [16] Wang *et al.*, "Enhancing adversarial text attacks on BERT models with projected gradient descent," 2024.
- [17] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP (System Demonstrations)*, 2020, pp. 38–45.